

Local File Inclusion (LFI)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una LFI ocurre cuando una pagina construye la ruta de un fichero a incluir usando un valor que viene del usuario (típicamente un parametro como `?page=`), sin comprobar que ese valor sea uno de los permitidos. El servidor entonces lee y a veces ejecuta el fichero que tu le pidas, no el que el programador esperaba.

El codigo hace algo como `include($_GET['page'].'.php')`. El programador asume que page siempre sera 'home' o 'about', pero nada lo obliga. Si metes `../../../../etc/passwd`, la funcion de inclusion resuelve esa ruta relativa y sirve el fichero. La causa raiz es confiar en la entrada del usuario para decidir que fichero abrir.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Sospecha LFI en cualquier parametro que parezca nombrar una vista, una plantilla, un idioma o un documento: page, file, view, template, lang, doc, include. Sobre todo si el valor aparece reflejado en la URL y el contenido de la pagina cambia segun ese valor.

Cómo identificarla

- Un parametro cuyo valor parece un nombre de fichero o de pagina (`?page=home`, `?file=report`).
- La respuesta cambia de forma coherente al pedir rutas: `../` te acerca a la raiz, un fichero inexistente da un error de include con la ruta.
- Mensajes de error de PHP que filtran rutas absolutas del servidor (failed to open stream).
- Puedes leer `/etc/passwd` (Linux) o `C:\Windows\win.ini` (Windows) via el parametro.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide `/etc/passwd` con suficientes `../` (o su ruta absoluta si el include no es relativo).

Evidencia esperada: Lineas que empiezan por `root:x:0:0` en la respuesta. Un error de include con una ruta absoluta tambien cuenta como confirmacion parcial.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s '$URL/?page=../../../../etc/passwd'
```

Objetivo y contexto: Prueba de lectura con un fichero que siempre existe. Los ../ de sobra suben hasta la raíz aunque no sepas la profundidad real del script.

Resultado esperado: Líneas root:x:0:0. Si aparecen, hay LFI confirmada. Un error de include con una ruta absoluta también es útil: te dice donde estás en el disco.

Capa 2: prueba documentada

```
curl -s '$URL/?page=php://filter/convert.base64-encode/resource=config'
```

Objetivo y contexto: Si el código añade .php y ejecuta el fichero, el wrapper php://filter te deja leer el FUENTE en base64 sin que se ejecute. Así lees configs con credenciales.

Resultado esperado: Un blob en base64: decodifícalo (base64 -d) para ver el código PHP. Busca ahí credenciales de BD, claves de API y rutas a otros ficheros jugosos.

Anatomía de los payloads

La primera prueba es `curl -s '$URL/?page=../../../../etc/passwd'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Líneas que empiezan por root:x:0:0 en la respuesta. Un error de include con una ruta absoluta también cuenta como confirmación parcial.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma la lectura con un fichero que siempre existe: pide /etc/passwd con suficientes ../ para llegar a la raíz.
- Si el código añade una extensión (.php), usa un PHP wrapper para leer el fuente en base64 sin que se ejecute, o busca truncar la extensión.

- Mapea que mas puedes leer: config con credenciales, claves SSH, logs. Cada fichero legible es una pista para la siguiente fase.
- Si controlas el contenido de algun fichero que el servidor luego incluye (un log, una sesion, un upload), la LFI se convierte en ejecucion: mira `lfi-a-rce`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El parametro no cambia el contenido de la pagina aunque le mandes `../` o rutas absolutas.
- La app usa una lista blanca de vistas validas y rechaza cualquier valor fuera de ella.
- El framework normaliza/sanea la ruta antes de tocar el filesystem (basename, whitelist).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.

4. Propón un control negativo y una mitigación causal.

Resumen

La web incluye un fichero cuyo nombre controlas tu; si no lo valida, lees ficheros del servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (lfi)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.